

Reviewer Pack — Minimal Rank of Noncommutative Crossed Products over Boolean...

Entry 3ea965353734 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 15:00:28 UTC

Minimal Rank of Noncommutative Crossed Products over Boolean Algebras vs AC0 Parity Circuit Size

Entry ID: 3ea965353734 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 15:00:28 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Geometry and Algebra (Crossed Products) **Field B** (complexity object): Complexity Theory: AC0 Parity Circuit Complexity

Statement:

For any Boolean algebra B , there exists a noncommutative crossed product with its maximal ideal that yields a minimal rank invariant, $\psi(B)$, such that for all circuits $C \in \text{AC0}$ computing PARITY on n inputs, $\psi(B) \geq c \cdot \log(\text{size}(C))$ for some constant c .

Rationale (proposer's reasoning):

Noncommutative crossed products provide a framework to encode algebraic structures with additional structure, which could potentially capture the complexity of Boolean functions in a way that escapes standard linear representations. This invariant might reveal non-trivial group actions on circuits, offering a new perspective on proving lower bounds for AC0 parity.

Taxonomy category: AC0_PARITY (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 513f06f9ebf6f493

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the computed minimal rank invariant $\psi(B)$ for each Boolean algebra B exceeds $c \cdot \log(\text{size}(C))$ by at least 3 standard deviations across all seeds, where $\text{size}(C)$ is the size of an AC0 PARITY circuit and c is a constant.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	SAFE	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.80	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - intitle:noncommutative geometry AND crossed products AND Boolean algebra AND AC0 parity circuit complexity - Minimal rank invariant AND noncommutative crossed products AND Boolean algebras AND AC0 circuit size - Parity circuit complexity AND AC0 AND noncommutative geometry AND Boolean algebra minimal rank

Top relevant hits considered: - [http://arxiv.org/abs/1310.1362v2] Complexity of linear circuits and geometry - [http://arxiv.org/abs/1711.08381v1] Symplectic, product and complex structures on 3-Lie algebras - [http://arxiv.org/abs/2211.07055v3] Geometric complexity theory for product-plus-power - [http://arxiv.org/abs/2108.04557v3] Brauer diagrams, modular operads, and a graphical nerve theorem for circuit algebras - [http://arxiv.org/abs/2202.13451v1] A note on infinite partitions of free products of Boolean algebras - [http://arxiv.org/abs/2202.02008v1] C*-algebras of generalized Boolean dynamical systems as partial crossed products - [http://arxiv.org/abs/1110.2998v1] Equivalent Quantum Circuits - [http://arxiv.org/abs/math/0409105v2] Tiling Parity Results and the Holey Square Solution

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_algebra(n):
        elements = [f"x{i}" for i in range(2**n)]
        operations = {}
        for a in elements:
            for b in elements:
                if a == b:
                    operations[(a, b)] = a
                else:
                    operations[(a, b)] = f"{a} v {b}"
        return operations

    def compute_crossed_product_rank(operations):
        # Simplified rank computation (not actual crossed product)
        return len(operations) ** 0.5

    def generate_ac0_circuit(n):
```

```

size = 2**math.ceil(math.log2(n))
circuit = []
for _ in range(size):
    gate = random.choice(["AND", "OR"])
    inputs = [random.choice([f"x{i}" for i in range(2**n)]) for _ in range(gate)]
    circuit.append((gate, inputs))
return circuit

def compute_circuit_size(circuit):
    size = 0
    for gate, _ in circuit:
        if gate == "AND":
            size += len(_)
        elif gate == "OR":
            size += len(_)
    return size

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    B = generate_boolean_algebra(n)
     $\psi_B$  = compute_crossed_product_rank(B)
    C = generate_ac0_circuit(n)
    size_C = compute_circuit_size(C)

    if size_C == 0:
        continue

    results.append( $\psi_B$  / math.log(size_C))

if not results:
    return {
        "metric_name": "minimal_rank",
        "metric_value": None,
        "instances_tested": 0,
        "conjecture_holds": False,
        "counterexample": "empty_circuit"
    }

mean = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))

return {
    "metric_name": "minimal_rank",
    "metric_value": mean,
    "instances_tested": len(n_values),
    "conjecture_holds": all(x >= mean + 3 * std_dev for x in results),
    "counterexample": ""
}

if __name__ == "__main__":
    seeds = [int(arg) for arg in sys.argv[1:]]
    if not seeds:
        seeds = [2**i - 1 for i in range(5, 6)] # Default to first few primes

```

```

results = []

for seed in seeds:
    trial_result = run_trial(seed)
    print(f"TRIAL: {trial_result}")

    if "metric_value" not in trial_result or trial_result["metric_value"] is None:
        continue

    results.append(trial_result["metric_value"])

mean = sum(results) / len(results)
std_dev = math.sqrt(sum((x - mean) ** 2 for x in results) / len(results))
support_fraction = sum(1 for x in results if x >= mean + 3 * std_dev) / len(results)

if all(x >= mean + 3 * std_dev for x in results):
    print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
elif any(x < mean + 3 * std_dev for x in results):
    first_failing_seed = seeds[results.index(next(x for x in results if x < mean + 3 * std_dev))]
    print(f"RESULT: FALSIFIED counterexample=\"not_enough_support\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE support_fraction_too_low")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbb3f912.py", line 96, in <module>
 trial_result = run_trial(seed)

^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbb3f912.py", line 60, in run_trial
 C = generate_ac0_circuit(n)

^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_dbb3f912.py", line 41, in generate_ac0_circuit
 inputs = [random.choice([f"x{i}" for i in range(2**n)]) for _ in range(gate)]

^^^^^^^^^^^^^^^^

TypeError: 'str' object cannot be interpreted as an integer

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating whether the conjecture is supported or falsified. | next: Investigate and fix the error in the test code to allow for proper evaluation of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 13

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15629
2	propose	ollama_remote	glm4:latest	0	0	13040
3	propose	ollama_remote	glm4:latest	0	0	14521
4	propose	ollama_remote	glm4:latest	0	0	14212
5	propose	ollama_remote	glm4:latest	0	0	12940
6	preregistration	ollama_remote	glm4:latest	0	0	17029
7	novelty	ollama_remote	glm4:latest	0	0	10541
8	novelty	ollama_remote	glm4:latest	0	0	10472
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15169
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9199
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7981
12	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10074
13	judge	ollama_remote	glm4:latest	0	0	38082

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 188887 ms total latency. Provider mix: {'ollama_remote': 13}

(full prompt+response transcripts available in *research/audit/3ea965353734.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/3ea965353734.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/3ea965353734.tar.gz* (if generated)